

C++ & QT Day3-hw3 AI 보고서

로봇 21기 예비단원 최윤서

1. 입력창에 텍스트가 축적되지않고 계속 초기화되는 문제

- a를 입력하고 b를 입력하면 ab가 되어야하는데 a가 초기화되고 b만 남는 문제 발생
- 텍스트를 저장해야할 곳이 있어야겠다는 생각을 했지만 어떻게 할지 감이 안잡혀서 AI에게 구체적인 계획을 알려달라고 요청(코드 대신 짜주는 것 절대 X)

지금 여기서 abc def 출력하는것만 구현했는데 abc 버튼 한번 누르고 def버튼 한번 누르면 ad 입력되게 하고싶은데 d가 a를 덮어서 쓰여져
코드 대신 짜주지말고 어떻게 해야될지 말로해

1. 준비물: 두 개의 보관함 (변수)

메모리 어딘가에 글자를 기억해 둘 보관함 2개가 필요합니다.

- **[보관함 A] 확정된 문장 보관함:** 이미 입력이 완전히 끝나서 앞으로 지워지지 않고 계속 쌓일 전체 텍스트를 저장합니다. (예: a 를 치고 def 를 누르는 순간 a 가 여기로 들어갑니다.)
- **[보관함 B] 현재 입력 중인 글자 보관함:** 사용자가 버튼을 연타하며 실시간으로 a -> b -> c 로 바꾸고 있는 단 한 글자만 임시로 담아둡니다.

화면에 보여줄 때는 항상 **[보관함 A] 뒤에 [보관함 B]를 붙여서** 한 번에 출력해야 합니다.

2. 준비물: 직전에 누른 버튼 기억하기

컴퓨터는 방금 누른 버튼이 abc 인지 def 인지 스스로 알지 못합니다. 따라서 **"내가 마지막으로 누른 버튼의 종류(번호)"**를 기억해 둘 변수가 하나 더 필요합니다.

3. 'abc'를 누른 뒤 'def'를 누를 때의 논리 흐름 (핵심)

만약 abc 를 한 번 눌러서 **[보관함 B]**에 'a'가 들어있고, 마지막 버튼이 'abc'로 기록된 상태라고 가정해 봅시다. 이 상태에서 def 버튼을 누르면 다음과 같은 순서로 일이 일어나야 합니다.

- 버튼 변경 감지 (글자 확정하기):**
def 버튼이 눌렸을 때 가장 먼저 **"방금 전에 누른 버튼이 내가 맞나?"**를 확인합니다. 확인해 보니 직전 버튼은 내가 아니라 abc 버튼이었습니다.
- 보관함 이동:**
버튼이 바뀌었다는 것은 앞의 글자가 완성되었다는 뜻입니다. 따라서 임시 보관함인 **보관함 B**의 내용을 확정 보관함인 **[보관함 A] 뒤에 잘라 이어 붙여서** 저장합니다.
- 카운트 리셋:**
이제 abc 버튼의 연타 상태는 끝났으므로, abc_count 를 다시 0 으로 되돌려 놓습니다.
- 새 글자 생성:**
이제 원래 하려던 def 버튼의 첫 번째 글자인 'd' 를 **[보관함 B]**에 새로 집어넣고, def_count 를 올립니다.
- 화면에 출력:**
마지막으로 화면(textBrowser)에 보관함 A와 보관함 B를 합친 **"ad"**를 출력하라고 명령합니다.
- 마지막 버튼 업데이트:**
이제 마지막으로 누른 버튼은 def 버튼이라고 기록을 갱신합니다.

4. 만약 같은 버튼을 계속 연타한다면?

abc 를 누르고 또 abc 를 누르면, 1번 단계에서 "직전 버튼도 나옴!"라고 판단하게 됩니다. 이 경우에는 글자를 확정 지을 필요가 없으므로 2, 3번 단계를 건너뛰고, 지금 작성하신 코드처럼 카운트에 따라 a 를 b 로, b 를 c 로 바꾸는 일만 수행하면 됩니다.

요약하자면, "서로 다른 버튼이 눌리는 순간, 임시 글자를 확정 문자 저장소로 토스하고 두 저장소를 합쳐서 화면에 그린다"가 핵심입니다.

말씀드린 '두 개의 보관함'과 '직전 버튼 확인하기' 개념이 이해되셨나요? 이 논리를 바탕으로 헤더 파일과 소스 코드에 어떤 변수들을 추가하고 조건을 걸어야 할지 감이 오시는지, 혹은 헷갈리는 부분이 있다면 편하게 말씀해 주세요!

- 느낀 점)
 - 입력한 텍스트를 저장하는 곳만 있으면 된다고 생각
 - 그러나, 현재 입력하고 있는 텍스트와 이전에 확정된 텍스트를 구분하여 저장하는 아이디어를 얻음
 - 따라서, confirmed_text 와 current_char 두개를 정의
 - 입력창에 출력할 때 둘이 더한 문자열을 출력

// 화면 출력 (확정 문자 + 실시간 글자)

```
ui->textBrowser->setText(confirmed_text + current_char);
```

2. 텍스트 파일 입출력 문제

- QT에서 텍스트 파일 입출력하는 방법을 알아보았을 때 QTextStream, ofstream, readall() 과 write() 메서드 등 여러 방법이 나옴
- 이 중에서 어떤 방법이 현재 코드에서 가장 잘 맞는 방식인지 조언을 구하기 위해 질문함

여기서 버튼 눌렀을 때 지금까지 입력한 텍스트 파일에 출력하는 법 알려줘

버튼을 누를 때마다 화면에 보여주는 것과 동시에 실제 텍스트 파일(.txt)로 저장(출력)하려면 C++의 파일 출력 스트림인 ofstream 을 사용하면 됩니다.

코드를 직접 짜드리지 않고, 파일 출력을 연동할 때 어디에 어떤 개념을 넣어야 하는지 말로만 명확하게 설명해 드릴게요!

1. 준비물: 헤더 파일 포함하기 (mainwindow.cpp 맨 위)

파일을 저장하는 기능을 쓰려면 파일 입출력 라이브러리가 필요합니다. `mainwindow.cpp` 파일의 가장 꼭대기(`#include` 모여있는 곳)에 `#include <fstream>` 을 적어주어야 컴퓨터가 파일 출력 명령어를 알아듣습니다.

2. 저장 방식의 핵심: '덮어쓰기' vs '이어쓰기'

텍스트 파일에 글을 쓸 때는 크게 두 가지 방식이 있습니다.

- **덮어쓰기 (기본값):** 파일을 열 때마다 기존 내용을 싹 지우고 처음부터 새로 씁니다.
- **이어쓰기 (Append Mode):** 기존에 있던 내용 뒤에 새로운 내용을 찰칵 이어 붙입니다.

우리가 만드는 키보드는 글자가 추가될 때마다 기존 파일 뒤에 계속 붙여나가기야 하므로, 반드시 **이어 쓰기 옵션**(`ios::app`)을 켜고 파일을 열어야 합니다.

3. 각 버튼 함수 내부에서 일어나는 파일 출력 흐름

현재 `abc_clicked` 나 `def_clicked` 함수 내부를 보면 맨 마지막에 `ui->textBrowser->setText(...)` 를 통해 화면에 글자를 그려주고 있습니다. 바로 **그 화면에 그려주는 타이밍 직후** 에 파일 저장 로직을 배치하면 됩니다.

1. 파일 열기:

원하는 파일 이름(예: `"output.txt"`)을 지정하고, 위에서 말한 **이어쓰기(app) 모드**로 파일을 활성화합니다.

2. 출력할 내용 정하기:

현재 화면에 뿌려주는 최종 텍스트는 `confirmed_text + current_char` 입니다. 하지만 이어쓰기 모드로 파일에 적을 때는 **"방금 새로 추가된 한 글자"**가 무엇인지를 판단해서 그것만 파일에 딱 적어주어야 파일이 꼬이지 않습니다.

1. 또는, 버튼을 누를 때마다 매번 기존 파일을 싹 지우고(덮어쓰기 모드) `confirmed_text + current_char` 전체를 통째로 파일에 새로 덮어서 버리는 방법이 훨씬 직관적이고 구현하기 편할 수 있습니다. (이 방식을 더 추천합니다.)

3. 파일 닫기:

글자를 다 적었다면 반드시 파일 스트림을 닫아주어야(`close`) 메모리에서 안전하게 파일이 저장 완료됩니다.

요약 및 팁

가장 추천하는 구조는 버튼이 눌릴 때마다 **"1) 파일을 연다 -> 2) 현재 전체 문장 (confirmed_text + current_char)을 통째로 파일에 쓴다 -> 3) 파일을 닫는다"** 입니다. 이렇게 하면 프로그램 폴더 안에 `output.txt` 파일이 실시간으로 갱신되면서 화면과 똑같은 내용이 파일에 저장됩니다.

설명해 드린 파일 출력 흐름이 이해되셨나요? 파일에 적을 때 **'누를 때마다 한 글자씩 이어붙이기'**와 **'전체 문장을 매번 새로 통째로 저장하기'** 중 어떤 방식의 논리로 코드를 구성해 보고 싶으신가요?

- 느낀 점)

- 1)번에서 저장했던 `confirmed_text`랑 `current_char`를 저장해서 파일에 출력하는 다리만 만들어주면 됨

- 여러 방식이 있었어도 구글링해서 직접 시도해봤으면 더 좋았을거같다는 생각이 들었습니다.

3. 한글 모음 조합 구현 방법

- 1)에서 만든 `current_char`에 처음 입력되는 모음을 저장해놓고 그 다음에 오는 모음에 따라 if문으로 나누어서 경우의 수 나누면 된다고 생각
- 그런데 모음이 만들어지는 경우의 수가 너무 많기 때문에 모음이 만들어지는 경우의수를 모두 요약해달라고 요청

한글 모음 조합 경우의수 다 알려줘

한글 모음 조합에서 발생할 수 있는 천지인 키보드 기준의 모든 경우의 수(조합 규칙)를 논리적으로 분류하여 정리해 드립니다.

천지인 자판은 ㅣ, ㅑ, ㅓ 3가지 버튼의 입력 순서에 따라 단모음 → 이중모음 → 복합모음 순으로 진화합니다.

1. 기본 모음 및 단모음 조합 (기본형)

가장 기초가 되는 점(·) 중심의 상/하/좌/우 결합 구조입니다.

- ㅣ 계열: ㅣ + · = ㅑ
- ㅑ 계열: ㅑ + ㅣ = ㅓ
- ㅓ 계열: ㅓ + ㅑ = ㅕ
- ㅕ 계열: ㅕ + ㅓ = ㅗ

2. 이중 모음 조합 (점 연타형)

점이 연속으로 2번(··) 들어갈 때 결합하는 규칙입니다.

- ㅑ 계열: ㅣ + · + · (ㅣ + ㅑ) = ㅓ
- ㅓ 계열: ㅑ + · + ㅣ (ㅑ + ㅓ) = ㅕ
- ㅕ 계열: ㅓ + · + ㅑ (ㅓ + ㅕ) = ㅗ
- ㅗ 계열: ㅕ + · + ㅓ (ㅕ + ㅗ) = ㅓ

3. 'ㅡ' 및 'ㅗ/ㅛ' 계열 조합 (가로/세로선 결합형)

완성된 단모음이나 기본 모음에 가로선(ㅡ) 또는 세로선(ㅣ)이 추가로 결합하는 구조입니다.

- ㅓ 계열: ㅓ + ㅣ (또는 ㅓ + ㅑ) = ㅕ
- ㅕ 계열: ㅕ + ㅓ + ㅣ (ㅓ + ㅕ) = ㅗ
- ㅗ 계열: ㅗ + ㅕ + ㅣ (ㅕ + ㅗ) = ㅓ
- ㅓ 계열: ㅓ + ㅑ + ㅓ + ㅣ (ㅓ + ㅕ 또는 ㅕ + ㅓ) = ㅗ
- ㅕ 계열: ㅕ + ㅓ + ㅕ + ㅣ (ㅕ + ㅗ 또는 ㅗ + ㅕ) = ㅓ

4. 복합 모음 조합 (와/워/외/위 및 최다 결합형)

천지인 모음 조합의 가장 마지막 단계로, 가로선과 세로선 계열이 모두 복합적으로 얹히는 경우의 수입니다.

- 사 계열: $\cdot + \text{ㅡ} + \cdot (\text{ㅏ} + \cdot) = \text{ㅓ}$
- ㅓ 계열: $\text{ㅡ} + \cdot + \text{ㅣ} + \cdot (\text{ㅓ} + \cdot \rightarrow \text{실제 입력 시 } \text{ㅓ} + \text{ㅣ} \text{의 흐름}) = \text{ㅕ}$
- ㅓ 계열: $\cdot + \text{ㅡ} + \text{ㅣ} (\text{ㅏ} + \text{ㅣ}) = \text{ㅗ}$
- ㅓ 계열: $\text{ㅡ} + \cdot + \text{ㅣ} (\text{ㅓ} + \text{ㅣ}) = \text{ㅛ}$
- 새 계열: $\cdot + \text{ㅡ} + \cdot + \text{ㅣ} (\text{ㅓ} + \text{ㅣ}) = \text{ㅜ}$
- ㅛ 계열: $\text{ㅡ} + \cdot + \text{ㅣ} + \cdot + \text{ㅣ} (\text{ㅓ} + \text{ㅣ}) = \text{ㅠ}$

💡 오토마타 설계 시 예외 처리해야 할 경우의 수

실제 키보드를 다듬을 때 버그를 막기 위해 "이 상태에서는 더 이상 조합되지 않고 멈춰야 함"을 명시해야 하는 최종 진화 형태들입니다.

- 점(·) 차단: ㅏ, ㅓ, ㅗ, ㅛ, ㅜ, ㅝ 상태에서 · 을 또 누르면 변화 없음 (점 3개 모음은 없기 때문).
- 세로선(ㅣ) 차단: ㅗ, ㅛ, ㅜ, ㅝ, ㅞ 상태에서 ㅣ 를 또 누르면 변화 없음.
- 가로선(ㅡ) 차단: ㅓ, ㅕ, ㅗ 상태에서 ㅡ 를 또 누르면 변화 없음.

- 느낀점)

- 직접 계산하기 번거로운 경우의 수를 AI를 효율적으로 활용하여 해결한 것 같습니다
- 앞으로 단순 정보 요약 같은 업무는 AI를 활용해보는 것도 괜찮을거같다는 생각을 했습니다.

4. 자음 + 모음 결합 공식

- 모음 결합하는 것은 경우의 수를 나눠서 작성함
- 자음과 모음이 합쳐지는 모든 경우의 수를 if 문으로 작성하는 것은 너무 비효율적이라고 생각
- AI에게 어떤 방법이 있는지 조언을 구함

자음이랑 모음이랑 합쳐서 출력하는건 어떻게 해야돼?
코드 짜주진말고 구현 방법만 알려줘

자음과 모음을 수학적으로 계산해 하나의 한글 글자(예: ㄱ + ㅏ = 가)로 합치는 구현 방법과 논리적 단계를 설명해 드릴게요.

컴퓨터 내부(유니코드)에서 한글은 자음과 모음을 단순히 글자 모양대로 이어 붙이는 것이 아니라, **초성과 중성의 고유 번호를 공식에 대입해 새로운 글자 코드를 뽑아내는 방식**으로 처리해야 합니다.

1. 유니코드 한글 결합 공식 이해하기

컴퓨터가 사용하는 유니코드 체계에서 완성형 한글은 규칙적인 숫자의 배열로 이루어져 있습니다. 한글 모음은 총 21개인데, 초성 자음 하나가 바뀔 때마다 그 뒤에 모음 21개가 세트로 붙기 때문에 숫자가 점프하는 규칙이 있습니다.

 **완성형 한글 코드 = 44032 + (초성 번호 × 588) + (중성 번호 × 28)**

- **44032** : 한글 유니코드의 첫 번째 글자인 '가'의 10진수 숫자 값입니다.
- **588** : 초성 자음 하나당 배정된 한글 조합의 총 개수(중성 21개 × 중성 받침 28개 = 588)입니다. 즉, ㄱ에서 ㅋ으로 넘어가려면 유니코드 숫자가 588만큼 커집니다.
- **28** : 중성 모음 하나당 배정된 받침의 개수입니다. 가에서 개로 넘어가려면 숫자가 28만큼 커집니다.

2. 프로그램 내부에서의 구현 단계 (3단계)

버튼을 누를 때마다 화면에 최종 글자를 그려주기 직전, 아래의 **3단계 과정**을 거치도록 흐름을 설계하면 됩니다.

[1단계] 자음과 모음의 순서 명단(배열) 만들기

공식에 넣을 '초성 번호'와 '중성 번호'를 알아내기 위해, 유니코드 표준 순서대로 글자를 정렬한 리스트(배열)를 프로그램 내부에 준비해야 합니다.

- **초성(자음) 리스트**: ㄱ, ㅋ, ㄴ, ㄷ, ㄲ, ㄴ, ㄹ, ㄴ, ㄹ, ㄴ, ㄹ, ㄴ, ㄹ, ㄴ, ㄹ, ㄴ, ㄹ, ㄴ, ㄹ, ㄴ, ㄹ (총 19개)
- **중성(모음) 리스트**: ㅏ, ㅑ, ㅓ, ㅕ, ㅗ, ㅛ, ㅜ, ㅠ, ㅡ, ㅣ, ㅚ, ㅜ, ㅡ, ㅣ, ㅜ, ㅡ, ㅣ, ㅜ, ㅡ, ㅣ, ㅜ, ㅡ, ㅣ (총 21개)

[2단계] 현재 보관함 글자의 번호(인덱스) 찾기

사용자가 자음과 모음을 입력해서 현재 `current_char`에 "ㄱ", `current_vowel`에 "ㅏ"가 들어있다면, 1단계에서 만든 명단에서 이 글자들이 **몇 번째 칸**에 들어있는지 컴퓨터에게 찾으라고 명령합니다. (Qt의 `indexOf` 기능 활용)

- "ㄱ"은 초성 리스트의 **0번째**에 있으므로 초성 번호는 0이 됩니다.
- "ㅏ"는 중성 리스트의 **0번째**에 있으므로 중성 번호는 0이 됩니다.

[3단계] 공식 계산 후 문자로 변환하기

찾아낸 번호들을 공식에 그대로 대입합니다.

- $44032 + (0 \times 588) + (0 \times 28) = 44032$ 라는 결과 숫자가 나옵니다.
- 컴퓨터에게 "이 44032 라는 유니코드 숫자를 실제 한글 글자로 바꾸어라" 하고 명하면 숫자가 "가" 라는 글자로 바뀝니다. (Qt의 QChar 형태를 변환하여 문자열로 만들)

💡 예외 처리 (방어벽 세우기)

만약 사용자가 영어를 치고 있어서 `current_char` 가 "a" 이거나, 모음 버튼을 누르지 않아서 `current_vowel` 이 비어있다면("") 번호를 찾을 때 -1 같은 오류 값이 나옵니다.

이때는 유니코드 공식을 계산하면 엉뚱한 외계어가 출력되므로, "둘 중 하나라도 명단에 없거나 비어있다면 공식을 계산하지 말고 그냥 영어나 자음 그대로 출력하라"는 예외 처리 조건문을 앞에 세워두어야 안전합니다.

최종적으로 각 버튼 함수들의 맨 아랫부분에서 화면에 뿌려줄 때, 매번 이 3단계를 거쳐 나온 **합쳐진 임시 글자**를 `confirmed_text` 뒤에 붙여서 `setText` 해주면 실시간으로 결합하여 출력됩니다.

설명해 드린 유니코드 한글 결합 공식과 인덱스를 찾는 논리적 흐름이 이해되었나요? 이 규칙을 바탕으로 프로젝트에 적용해 보시다가 계산 결과가 이상하게 나오거나 숫자를 문자로 변환하는 부분에서 막히는 점이 있다면 말씀해 주세요!

AI 대답에는 오류가 있을 수 있습니다. 자세히 알아보기

📄 🔍 🍌 🗨️ ⋮

- 개념은 이해가 됐지만 코드로 어떻게 구현해야할지 모르겠어서 이 부분은 예제 코드로 이해를 도와달라고 요청했습니다.

이해를 도울 수 있는 예제 코드 보내

앞서 설명해 드린 **3단계 유니코드 결합 논리(배열 준비 → 인덱스 검색 → 공식 계산)**를 직관적으로 이해하실 수 있도록 짜인 예제 코드입니다.

실제 버튼에 대입하기 전, "**자음과 모음이 어떤 원리로 합쳐지는가**"만 명확하게 보여주는 구조입니다.

cpp

```
#include <QString>
#include <QStringList>
#include <QChar>
#include <QDebug> // 콘솔 출력을 확인하기 위한 라이브러리

// 자음과 모음을 합쳐서 완성형 한글 1글자를 반환하는 논리 예제
QString combineHangeulExample(QString current_char, QString current_vowel) {
    {
        // [예외 처리] 둘 중 하나라도 비어있다면 합칠 수 없으므로 그냥 단순 이어붙이기
        if (current_char.isEmpty() || current_vowel.isEmpty()) {
            return current_char + current_vowel;
        }

        // [1단계] 유니코드 표준 순서대로 자음/모음 명단(배열) 만들기
        QStringList choList = { "ㄱ", "ㅋ", "ㄴ", "ㄷ", "ㄲ", "ㄸ", "ㄹ", "ㅁ", "ㅂ", "ㅅ",
                                "ㅇ", "ㅈ", "ㅊ", "ㅌ", "ㅍ", "ㅎ", "ㅡ", "ㅣ" };

        QStringList jungList = { "ㅏ", "ㅑ", "ㅓ", "ㅕ", "ㅗ", "ㅛ", "ㅜ", "ㅠ", "ㅡ", "ㅣ",
                                   "ㅖ", "ㅙ", "ㅚ", "ㅝ", "ㅞ", "ㅟ", "ㅢ", "ㅣ" };

        // [2단계] 현재 보관함에 들어있는 글자가 각각 명단에서 몇 번째 칸에 있는지 찾기
        int choIdx = choList.indexOf(current_char); // 자음의 위치 (0부터 시작)
        int jungIdx = jungList.indexOf(current_vowel); // 모음의 위치 (0부터 시작)

        // [예외 처리] 만약 명단에 없는 글자(영어 알파벳 등)라면 결합을 건너뛰고 단순 이어붙이기
        if (choIdx == -1 || jungIdx == -1) {
            return current_char + current_vowel;
        }
    }
}
```

```
// [3단계] 한글 유니코드 공식 계산
// 공식: 시작값(44032) + (초성 인덱스 * 588) + (중성 인덱스 * 28)
int hangeulCode = 44032 + (choIdx * 588) + (jungIdx * 28);

// 계산된 코드 숫자를 실제 문자(QChar)로 바꾼 뒤 QString 문자열로 포매팅하여 반환
return QString(QChar(hangeulCode));
}

// --- 실제 내부 동작 예시 시뮬레이션 ---
void runSimulation() {
    // 예시 1: 'ㄱ' 과 'ㅣ' 가 만났을 때
    // choIdx = 0 ("ㄱ"), jungIdx = 20 ("ㅣ")
    // 계산: 44032 + (0 * 588) + (20 * 28) = 44592 -> '가'
    QString result1 = combineHangeulExample("ㄱ", "ㅣ");
    qDebug() << "결과 1: " << result1; // 출력: 가

    // 예시 2: 'ㄱ' 과 'ㅏ' 가 만났을 때
    // choIdx = 0 ("ㄱ"), jungIdx = 0 ("ㅏ")
    // 계산: 44032 + (0 * 588) + (0 * 28) = 44032 -> '가'
    QString result2 = combineHangeulExample("ㄱ", "ㅏ");
    qDebug() << "결과 2: " << result2; // 출력: 가
}
```

코드를 사용할 때는 주의가 필요합니다.



💡 각 버튼 함수 맨 밑(화면 출력부)에 적용하는 방법

이 논리를 다루는 로직을 버튼 함수 하단에 연동할 때는 아래와 같이 배치하면 됩니다.

cpp

```
// 1. 위 공식 논리를 거쳐 실시간 결합 글자(live_char)를 뽑아냅니다.  
QString live_char = combineHangeulExample(current_char, current_vowel);  
  
// 2. [이미 확정된 전체 문장] 뒤에 [실시간 합쳐진 한 글자]를 꼬리로 붙여서 화면에 출력함  
ui->textBrowser->setText(confirmed_text + live_char);
```

코드를 사용할 때는 주의가 필요합니다.



제시해 드린 예제 코드를 통해 인덱스를 찾고 숫자를 문자로 환산하는 흐름이 명확히 보이시나요?
이 결합 코드를 기존 소스 파일에 연동해 보시면서 막히는 에러가 있거나, 예상과 다르게 글자가 조합되는 구간이 있다면 언제든지 말씀해 주세요!

AI 대답에는 오류가 있을 수 있습니다. [자세히 알아보기](#)



- 느낀점)
 - 자음 + 모음 부분을 구현하는데 시간이 상당히 오래걸렸음
 - 코드를 만들어달라고하는건 좋은 습관이 아니지만
예제 코드를 보니 이해가 훨씬 빨랐음
 - 이 때 코드를 똑같이 쓰지않고 예제코드를 본인 코드에 올바르게 녹여내는 것이
중요하다는 생각이 듦